

EECS 482

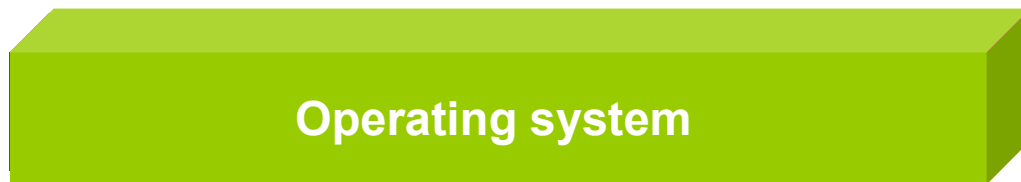
Introduction to operating systems

Locks; thread-safe queue

Peter Chen
University of Michigan

Higher-level synchronization

- Raise the level of abstraction to make life easier for programmers



Higher-level synchronization
operations
(**lock**, condition variable, semaphore)



Atomic operations
(**atomic load**, **atomic store**, etc.)

Locks (mutexes)

- A lock prevents another thread from entering a critical section
 - *“Lock fridge while checking milk status and shopping”*
- Two operations
 - **lock()**: wait until lock is free, then acquire it

```
while (1) {  
    if (lock is free) {  
        acquire lock  
        break  
    }  
}
```


 - **unlock()**: 
- **Problems?** Hint: why was the note in *Too much milk* (solutions #1 and #2) not a good lock?

Locks (mutexes)

- How to use locks
 - Lock is initialized to free
 - Thread should acquire lock before entering critical section
 - Thread that acquired lock should release lock when done with critical section

```
lock()  
<critical section>  
unlock()
```

- Thread can be running or blocked
- All synchronization involves waiting

Locks make “Too much milk” easy!

mutex milk;

Peter

```
milk.lock()  
if (noMilk) {  
    buy milk  
}  
milk.unlock()
```

Janet

```
milk.lock()  
if (noMilk) {  
    buy milk  
}  
milk.unlock()
```

Efficiency

- But this prevents Janet from doing things while Peter is buying milk

Peter

milk.lock()

if (no milk) {
 buy milk

}

milk.unlock()

Janet

milk.lock()

if (no milk) {
 buy milk

}

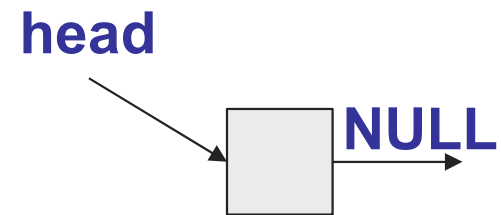
milk.unlock()

- How to minimize the time the lock is held?

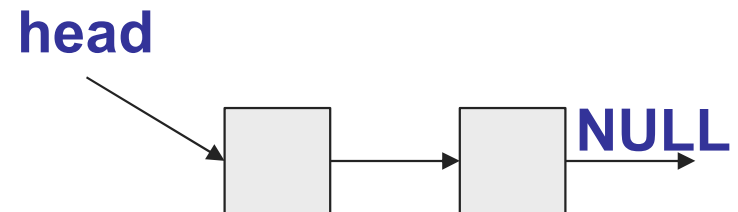
Thread-safe queue

```
struct node {  
    int data  
    struct node *next  
}
```

Empty list

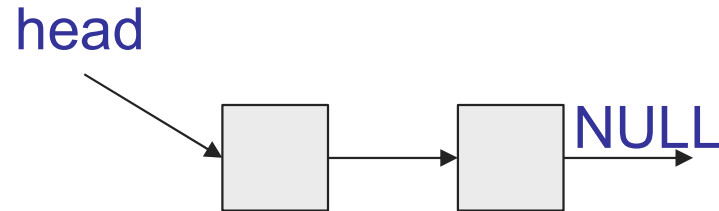


List with one node



Thread-safe queue

```
enqueue(node* new_element) {  
    node* ptr;  
    // find tail of queue  
    for (ptr = head; ptr->next != NULL; ptr = ptr->next) {}  
  
    // add new element to tail of queue  
    ptr->next = new_element;  
    new_element->next = NULL;  
}
```



```
node* dequeue() {  
    node* ptr = NULL;  
    // if something on queue, then remove it  
    if (head->next != NULL) {  
        ptr = head->next;  
        head->next = head->next->next;  
    }  
    return(ptr);  
}
```

Problems if two threads manipulate queue at same time?

Thread-safe queue with locks

```
mutex queueMutex;
```

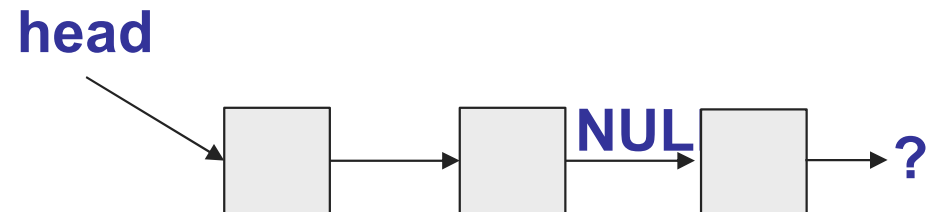
```
enqueue(node* new_element) {  
    queueMutex.lock();  
    node* ptr;  
    // find tail of queue  
    for (ptr=head; ptr->next != NULL; ptr = ptr->next) {}  
  
    // add new element to tail of queue  
    ptr->next = new_element;  
    new_element->next = NULL;  
    queueMutex.unlock();  
}  
  
node* dequeue() {  
    queueMutex.lock();  
    node* ptr = NULL;  
    // if something on queue, then remove it  
    if (head->next != NULL) {  
        ptr = head->next;  
        head->next = head->next->next;  
    }  
    queueMutex.unlock();  
    return(ptr);  
}
```

Can enqueue() unlock anywhere?

```
mutex queueMutex;
```

```
enqueue(node* new_element) {  
    queueMutex.lock();  
    node* ptr;  
    // find tail of queue  
    for (ptr=head; ptr->next != NULL; ptr = ptr->next) {}  
  
    // add new element to tail of queue  
    ptr->next = new_element;  
    new_element->next = NULL;  
    queueMutex.unlock();  
}
```

```
node* dequeue() {  
    queueMutex.lock();  
    node* ptr = NULL;  
    // if something on queue, then remove it  
    if (head->next != NULL) {  
        ptr = head->next;  
        head->next = head->next->next;  
    }  
    queueMutex.unlock();  
    return(ptr);  
}
```



Thread-safety invariants

- **When can enqueue() unlock?**
 - Only when queue is in a safe state
- “Safe state” = **invariant**
 - Condition that is “always” true for the linked list
 - Example: each node appears exactly once; last node points to NULL
- **Is invariant ever allowed to be false?**
 - Invariant may be broken only when the lock is held
 - Only the thread holding the lock may break the invariant
 - ➔ Hold lock whenever you’re manipulating shared data
- **What if you are only reading the data?**

What does acquiring a lock accomplish?

- Before acquisition: acquiring a lock stops **this thread** from entering a critical section if another thread currently holds the lock
- After acquisition: acquiring a lock stops **other threads** from entering their critical section

Does this work?

```
enqueue(node* new_element) {  
    lock  
    node* ptr;  
    // find tail of queue  
    for (ptr=head; ptr->next != NULL; ptr = ptr->next) {}  
    unlock  
    lock  
    // add new element to tail of queue  
    ptr->next = new_element;  
    new_element->next = NULL;  
    unlock  
}
```

Beware of assumptions that were verified in a prior lock region.

Fine-grained locking

- Instead of one lock for entire queue, use one lock per node
 - Why would you want to do this?
- Lock each node as the queue is traversed, then release as soon as it's safe, so other threads can also access the queue



1. lock A
2. read A
3. unlock A
4. lock B
5. read B
6. unlock B

What problems could occur?

How to fix?

1. lock A
2. read A
3. unlock A
4. lock B
5. read B
6. unlock B

- Hand-over-hand locking
 - Lock next node before releasing last node
 - Use in Project 4

Administration

- Project 1
 - Most material covered (condition variables covered next)
- Form project groups

What if you wanted dequeue() to wait if queue is empty?

```
node* dequeue() {  
    // wait for queue to be non-empty  
    while(head->next == NULL) {}  
  
    queueMutex.lock();  
  
    // remove element  
    ptr = head->next;  
    head->next = head->next->next;  
  
    queueMutex.unlock();  
    return(ptr);  
}
```

- Problems?

What if you wanted dequeue() to wait if queue is empty?

```
node* dequeue() {
    queueMutex.lock();
    // wait for queue to be non-empty
    while(head->next == NULL) {
        queueMutex.unlock();
        queueMutex.lock();
    }
    // remove element
    ptr = head->next;
    head->next = head->next->next;
    queueMutex.unlock();
    return(ptr);
}
```

- **Correct**
 - Lock is held continuously between seeing node on queue and dequeuing it
 - Allows another thread to enqueue while dequeue() is waiting
- But uses **busy waiting**

Waiting without busy waiting

- Waiting dequeuer adds itself to set of waiting dequeuers, then “goes to sleep”

```
if (queue is empty) {  
  
    add myself to waiting set  
  
    go to sleep and wait for wakeup  
}
```

- Enqueuer wakes up sleeping dequeuer

Waiting without busy waiting

enqueue ()

```
lock
add new item to tail of queue
if (dequeuer is waiting) {
    remove waiting dequeuer from waiting set
    wake up dequeuer
}
unlock
```

dequeue ()

```
lock
if (queue is empty) {
    → unlock?
    add myself to waiting set
    → unlock?
    go to sleep and wait for wakeup
    lock?
}
remove item from queue
unlock
```

What's wrong here?

Two types of synchronization

- **Mutual exclusion (“not at the same time”)**
 - Ensures that only one thread is in critical section
 - Provided by locks
- **Ordering constraints (“before/after”)**
 - One thread waits for another to do something
 - E.g., dequeuer must wait for enqueueer to add something to queue
 - Provided by condition variables

Condition variables

- Enable thread to sleep inside a critical section by calling `cv::wait`
 - Release lock
 - Put thread onto waiting set
 - Go to sleep

`atomic`

 - After being woken, acquire lock when it's free
- Each condition variable has a `set of waiting threads`
 - These threads are “waiting on that condition”
- Each condition variable is associated with a lock
- `cv::signal` wakes one waiting thread; `cv::broadcast` wakes all waiting threads

Condition variables

- Condition variable = a place for threads to wait
 - Threads in `cv::wait` are waiting on that condition
- **wait(mutex)** atomic
 - Release **mutex**, add thread to waiting set, go to sleep After waking, re-acquire **mutex** (blocking if needed).
 - Each condition variable is associated with a lock
 - Must hold lock when calling `wait()`
 - Invariant must be true when releasing lock
- **signal()** and **broadcast()**
 - Wake up one thread (signal) or all threads (broadcast) that are waiting on this condition variable
 - Need not be holding lock
 - If no thread is waiting, signal/broadcast does nothing